



nextwork.org

Create Kubernetes Manifests



Kehinde Abiuwa

The screenshot shows an AWS console terminal window with the following content:

```
aws | Search | [Option+S] | Europe (Stockholm) | Account ID: 5112-3943-1868 | kehindebiuwa-IAM-Admin
```

```
GNU nano 8.3 flask-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
```

At the bottom of the terminal, there is a status bar with the following information:

```
i-Odbb84502867a6a07 (nextwork-eks-instance)
PublicIPs: 51.20.128.54 PrivateIPs: 172.31.20.159
```



Introducing Today's Project!

In this project, I will set up a Kubernetes Deployment for the backend, create a Service to expose it to users, and dig into the manifest details to learn more, because these manifests define how the app runs and scales in Kubernetes, how users can reach it, and understanding them makes our deployment reliable and secure.

Tools and concepts

I used Amazon EKS, eksctl, kubectl, Amazon ECR, and Git/GitHub to create an EKS cluster (on EC2 nodes), build and push the backend image to ECR, and apply Deployment/Service YAML to run and expose the app. Key concepts include using manifests to declare desired state (Deployments & Services), label selectors to route traffic to Pods, replicas for high availability, rolling updates/rollbacks, and how NodePort/LoadBalancer types make the service reachable.

Project reflection

I chose to do this project today because it directly strengthens my portfolio for cloud/DevOps roles



This project took me approximately 40 minutes. The most challenging part was getting AWS auth right so nodes could pull from ECR and making sure the Service selector/ports matched the Deployment labels. My favourite part was seeing Kubernetes spin up 3 replicas and expose them via the Service, plus annotating the YAML so it's portfolio-ready and easy to explain.



Manifest files

Kubernetes manifests are declarative YAML files that describe the desired state of your resources (e.g., Deployments, Services, ConfigMaps, Secrets) so the cluster can create and maintain them. Manifests are helpful because they make deployments repeatable and version-controlled, enable safe rollouts/rollbacks, document configuration in one place, support self-healing by letting Kubernetes reconcile drift, and let you review changes (diffs) before applying them.

A Kubernetes deployment manages the desired state and lifecycle of a set of identical Pods—keeping the requested replica count running, selecting Pods by labels, performing rolling updates/rollbacks, and replacing failed Pods to maintain the spec. The container image URL in my Deployment manifest tells Kubernetes exactly which image to run and where to pull it from (registry/repository with a tag or digest), so each node can fetch that image and start/update the Pods accordingly.





Service Manifest

A Kubernetes Service exposes a stable virtual IP/DNS name and port for a set of Pods and load-balances traffic across them, independent of Pod restarts or scaling; it can be ClusterIP, NodePort, or LoadBalancer. You need a Service manifest to declare the selector, ports (port/targetPort), and type so Kubernetes can create that stable endpoint, route traffic to healthy Pods, and make your app discoverable and reliably reachable

My Service manifest sets up a NodePort Service named nextwork-flask-backend that selects Pods with the label app: nextwork-flask-backend, exposes port 8080/TCP and forwards to targetPort 8080—and since nodePort isn't specified, Kubernetes will auto-assign one so the app is reachable at <any node IP>:<auto nodePort>.





Deployment Manifest

Annotating the Deployment manifest helped me understand what each field does—replicas = how many Pods, selector = which Pods, image/port = what runs and where—because the comments turned the YAML into clear, step-by-step notes I can use to debug, review, and explain my setup.

One part of the Deployment manifest I still want to know more about is the rollout strategy, because it controls how updates are rolled out, balancing zero-downtime, capacity, and risk during deploys.

```
---
apiVersion: apps/v1 # Specifies the API version for this Deployment
kind: Deployment # This is a Deployment object
metadata: # Standard object metadata for this resource (contains name, namespace)
  name: nextwork-flask-backend # Unique Deployment name within the namespace
  namespace: default # The namespace to create this Deployment in; 'default' is used when none is specified
spec: # Deployment specification - declares the desired state
  replicas: 3 # Run 3 identical Pods for High Availability and load-balancing; Kubernetes maintains this count
  selector: # Which Pods this Deployment manages; MUST match the Pod template's labels
    matchLabels:
      app: nextwork-flask-backend
  template: # Pod template; new/updated Pods are created from this spec
    metadata:
      labels:
        app: nextwork-flask-backend
    spec: # Pod specification
      containers:
        - name: nextwork-flask-backend
          image: 511239431868.dkr.ecr.eu-north-1.amazonaws.com/nextwork-flask-backend:latest
          ports:
            - containerPort: 8080
```

i-0dbb84502867a6a07 (nextwork-eks-instance)
PublicIPs: 51.20.128.54 PrivateIPs: 172.31.20.159



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

